# PROJETO: FinStart




**Curso: Ciência de Dados e IA**

**Disciplina: PI: Programação Web Código: 2181161**

**Turma: 101**

**Professor: Fernando Luiz de Almeida Silveira**

**Nome do Grupo: FinStart**

**Gabriel Polli Perin – RA: 25007439**

**Yuri Kikuchi – RA: 25007272**

**Enzo Cayres Ferreira – RA: 25000144**

**Eduardo dos Santos Fernandez – RA: 25000693**

**Paulo Henrique Scavariello Guimarães – RA: 25017738**

**Campinas, 2º semestre de 2026**

# 1. Ideação do Projeto

**Correção e complemento da 2ª entrega — revisão de 28/10/2026.**

A FinStart é uma plataforma gratuita de educação financeira para jovens que estão começando a organizar renda, despesas e objetivos. Reúne conteúdos educativos, orçamento, projeções de juros compostos e acompanhamento de metas. O problema abordado é a dificuldade de transformar conceitos financeiros em decisões e hábitos de planejamento.

O público inicialmente definido é de jovens de 16 a 30 anos e pessoas com pouco conhecimento sobre finanças. A aplicação não verifica idade nem coleta data de nascimento. A proposta mantém o eixo aprender, simular e planejar, sem movimentar dinheiro ou recomendar produtos financeiros individuais.

A ODS 4 é a contribuição principal, pelo acesso a conhecimento e habilidades de planejamento. A relação com a ODS 8 é indireta: compreender serviços e organizar recursos pode apoiar decisões financeiras. A plataforma não comprova geração de renda, empregos ou impacto econômico mensurável.

**Histórico e escopo desta revisão**

A primeira proposta concentrava-se no front-end e em armazenamento no navegador. Após o retorno do professor, a arquitetura foi revista para Full Stack. A segunda entrega formalizou HTML/CSS/JavaScript, Node.js/Express, API REST, MySQL e autenticação JWT. Esta revisão apresenta a implementação dessas camadas e as correções posteriores; não atribui os recursos atuais à entrega original de 01/10/2026.

A versão atual utiliza MySQL local em uma instância exclusiva da FinStart. O SQLite usado durante o desenvolvimento foi mantido como cópia de segurança e alternativa de execução. As evidências mostram funcionamento nos cenários verificados, sem afirmar que testes finitos garantem ausência de qualquer falha.

# 2. Protótipo da Solução

O conjunto de telas foi ampliado para representar a jornada do MVP. Início apresenta as ferramentas; Aprender oferece leitura por assunto e nível; Orçamento organiza a renda; Juros e Comparar apresentam cenários; Metas registra objetivos; Meu painel reúne dados da conta; Sobre explica a proposta. O painel aparece no fim do menu, à direita no desktop.

A identidade visual permanece preta e dourada. O menu recolhe em telas pequenas, os campos têm rótulos, o foco de teclado é visível e os gráficos possuem tabelas equivalentes. Estados vazios, avisos de erro e confirmação de operações orientam o uso.

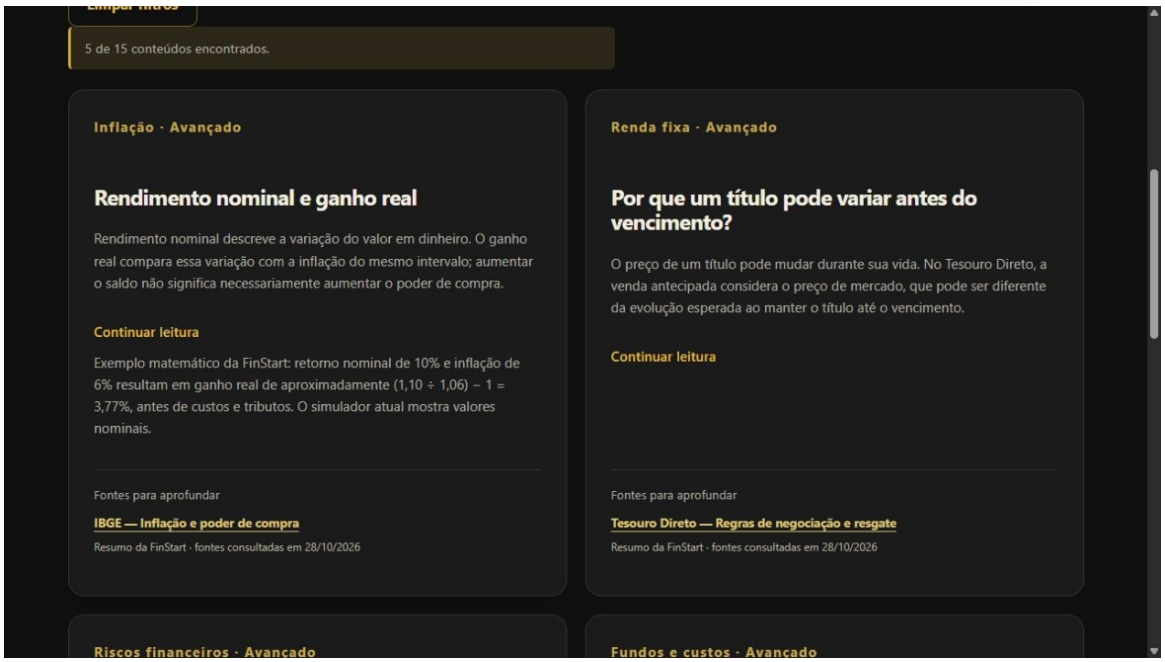



Figura 1 — Biblioteca com filtro avançado, referências oficiais e leitura expansível. Captura da revisão de 28/10/2026.

## Fluxos centrais implementados

Visitante: ler conteúdo → simular orçamento ou juros → comparar hipóteses. Conta autenticada: entrar → salvar simulação → criar ou editar meta → registrar saldo → consultar gráfico e painel. Os dados financeiros persistem no servidor; a preferência de filtro pode ficar no navegador.

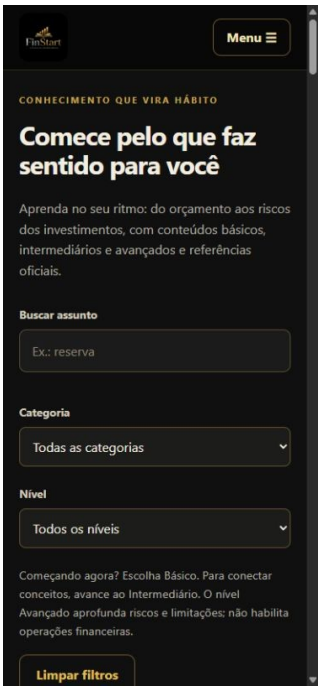



Figura 2 — Biblioteca em viewport de 390 px. Teste em navegador; não representa teste em aparelho físico.

# 3. Planejamento do Projeto

| Prioridade | Item e critério de conclusão |
|---|---|
| P1 | Conta e SQL: cadastro, login, logout e isolamento dos registros verificados. |
| P1 | Ferramentas: orçamento, projeções, comparação e metas com gravação e consulta. |
| P1 | Progresso: prazo, aporte mensal necessário, atualização do saldo e histórico. |
| P2 | Aprendizagem: 15 conteúdos com fontes, busca, categorias e três níveis. |
| P2 | Qualidade: testes de API, responsividade, documentação e preservação de dados. |
| P2 | Entrega: revisão pelo grupo, publicação e apresentação oral. |

### Kanban e acompanhamento

Concluído: fluxos do MVP, integração Express/API/JWT/SQL, MySQL exclusivo, biblioteca ampliada e verificações descritas na seção 6. Em andamento: conferência final do pacote e preparação da apresentação pelo grupo. A fazer: publicação no repositório com acesso de colaborador e envio no Canvas. O quadro datado está em docs/kanban.json.

O quadro registra o trabalho observável desta revisão. Não há evidência suficiente para reconstruir movimentações, reuniões ou sprints anteriores; por isso, esses eventos não são apresentados como realizados. Para os próximos ciclos, cada item deve ter responsável, critério de conclusão e evidência.

### Divisão prevista na segunda entrega

Paulo: autenticação e juros; Eduardo: dados e metas; Enzo: API; Gabriel: progresso e gráficos; Yuri: conteúdos e acompanhamento. Essa distribuição é a prevista no planejamento e precisa ser confirmada pelo grupo para a apresentação; não comprova autoria individual das alterações atuais.

# 4. Levantamento de Requisitos

| ID | Requisito e situação |
|---|---|
| RF01 | Cadastrar, autenticar e encerrar sessão: implementado. |
| RF02 | Consultar conteúdos por assunto, categoria e nível: implementado. |

| ID | Requisito e situação |
|---|---|
| RF03 | Calcular e salvar orçamento 50/30/20: implementado. |
| RF04 | Projetar juros compostos e mostrar evolução em gráfico/tabela: implementado. |
| RF05 | Comparar cenários e salvar resultados: implementado com taxas hipotéticas. |
| RF06 | Criar, consultar, editar e excluir metas com prazo: implementado. |
| RF07 | Registrar progresso e calcular aporte mensal necessário: implementado. |
| RF08 | Consultar painel e históricos exclusivos da conta: implementado. |
| RNF01 | Separar front-end, API, regras de domínio e persistência SQL. |
| RNF02 | Validar entradas no servidor e restringir operações ao proprietário. |
| RNF03 | Aceitar uso responsivo, teclado e leitura tabular dos gráficos. |
| RNF04 | Manter configuração privada fora do Git, com backup e instruções de execução. |

### Regras de negócio

O orçamento distribui a renda em 50% necessidades, 30% desejos e 20% futuro como referência educativa. Valores monetários persistem em centavos inteiros. Juros usam taxa mensal constante e aportes ao fim de cada mês. A comparação converte taxas anuais hipotéticas em taxas mensais equivalentes, com os mesmos valores e prazo.

A meta contém valor-alvo, saldo, prazo e data inicial. O saldo não pode ser negativo nem superar o alvo. A atualização e o registro de progresso ocorrem na mesma transação. O aporte necessário divide o valor restante pelos meses restantes, sem supor rendimento. Registros privados exigem sessão válida e pertencimento à conta.

As projeções não descontam inflação, taxas ou tributos. Recuperação de senha, verificação de e-mail, produtos reais, cotações e hospedagem pública não estão implementados nesta versão.

## 5. Desenvolvimento do Projeto

### Arquitetura e organização

Navegador HTML/CSS/JavaScript → requisições fetch/JSON → Node.js 24 e Express → controllers → adaptador MySQL → banco relacional. O navegador não

recebe credenciais SQL nem lê arquivos do banco. A arquitetura é um monólito em camadas, adequado à demonstração e à organização atual do projeto.

server/index.js organiza as rotas; controllers.js executa as operações; auth.js trata autenticação; mysql.js e database.js oferecem persistência; dominio.js e financas.js concentram regras de cálculo. database/ reúne os scripts SQL, tests/ reúne verificações e docs/ contém relatório e evidências. Scripts de inicialização, configuração e backup são utilitários operacionais necessários.

## Modelo de dados atualizado

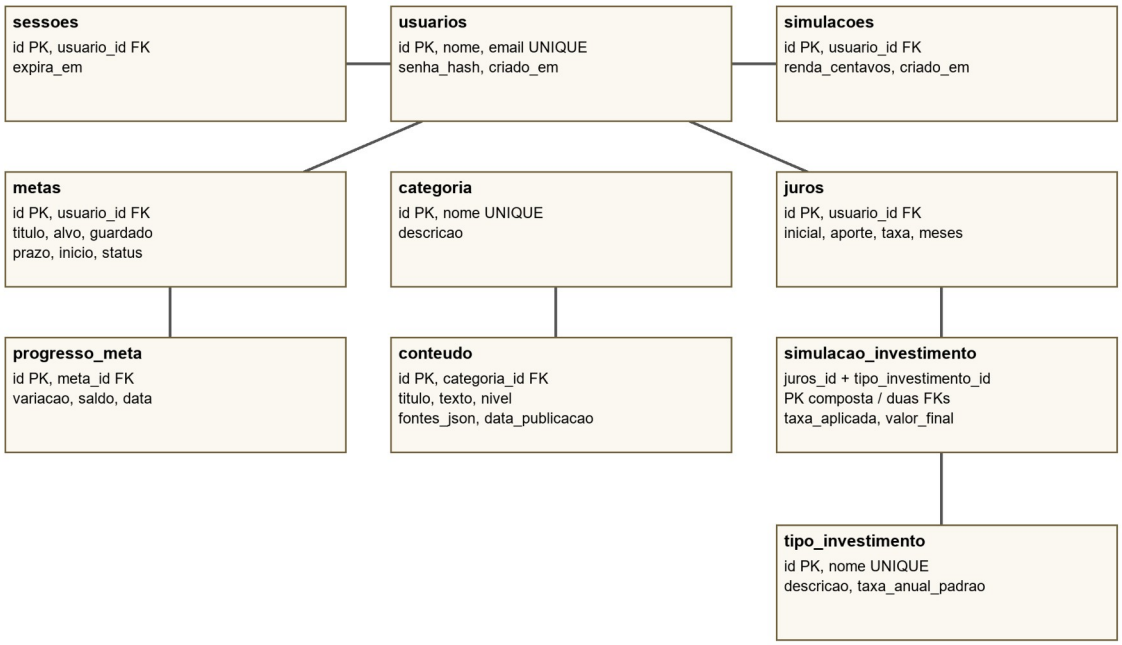



Figura 3 — DER físico simplificado. Cada ligação representa relação 1:N; simulacao_investimento possui chave primária composta e implementa a associação entre projeção e cenário.

| Tabela | Função e relacionamento |
|---|---|
| usuarios | Conta; email único e senha_hash. Pai de sessões, orçamentos, metas e juros. |
| sessoes | Sessão revogável; usuario_id referencia usuarios. |
| simulacoes | Orçamentos; usuario_id referencia usuarios. |
| metas | Objetivo, alvo, saldo, prazo, início e status; usuario_id referencia usuarios. |
| progresso_meta | Variação e saldo por registro; meta_id referencia metas. |
| juros | Plano de projeção; usuario_id referencia usuarios. |

| Tabela | Função e relacionamento |
|---|---|
| categoria | Classificação dos conteúdos. |
| conteudo | Texto, nível, referências e data; categoria_id referencia categoria. |
| tipo_investimento | Cenário didático com taxa anual declarada. |
| simulacao_investimento | Resultado por juros_id e tipo_investimento_id; duas FKs e PK composta. |

O catálogo físico contém 10 tabelas, 8 chaves estrangeiras e 51 colunas. Há chaves primárias, unicidade, checks e exclusão em cascata nas relações de propriedade. O script executável é database/schema.mysql.sql; database/consultas-demonstracao.mysql.sql permite inspecionar tabelas, colunas e relacionamentos sem alterar registros.

## Conciliação com o MER anterior

As entidades usuario e meta usam os nomes físicos usuarios e metas. A projeção planejada como simulacao é armazenada em juros, enquanto simulacoes identifica o orçamento. As entidades categoria, conteudo, tipo_investimento, simulacao_investimento e progresso_meta foram implementadas. Sessões e orçamento complementam as oito entidades do planejamento. Novos atributos operacionais incluem prazo, status e referências do conteúdo; nascimento foi omitido por minimização de dados.

## Conexão MySQL e migração

O MySQL Server existente estava ativo, mas sua senha administrativa não estava disponível. Foi criada uma instância exclusiva da FinStart, acessível somente em 127.0.0.1:3308, preservando o serviço anterior. A aplicação usa finstart_app restrito ao banco finstart; há um usuário separado de leitura para inspeção no Workbench.

Antes da troca, foi gerado backup do SQLite. A migração copiou registros mantendo IDs, hashes de senha e relacionamentos, comparou todos os campos originais e confirmou as contagens antes do commit. O catálogo educativo foi atualizado em seguida. Mudar apenas DB_DRIVER não realiza essa transferência: o procedimento utilizado está em scripts/configurar-mysql-local.js.

## Rotas e segurança

| Rotas | Finalidade |
|---|---|
| GET /api/conteudos e /api/tipos | Catálogo público e cenários. |
| POST /api/calcular e /api/comparar | Cálculos validados no servidor. |
| POST /api/cadastro e /api/login | Criação de conta e autenticação. |
| GET /api/me; POST /api/logout | Conta atual e revogação da sessão. |
| GET/POST /api/simulacoes e /api/juros | Orçamentos e projeções por usuário. |
| GET/POST /api/comparacoes | Comparações persistidas por conta. |
| GET/POST /api/metas; PUT /api/metas/:id | Criação, consulta e edição de metas. |
| GET /api/metas/:id/progresso | Histórico protegido de saldo. |
| DELETE /api/{recurso}/:id | Exclusão autorizada dos recursos suportados. |
| GET /api/saude | Teste da conexão real com SQL. |

Senhas usam scrypt com salt. JWT possui assinatura HS256, expiração e verificação no servidor, associado a sessão revogável. O cookie é HttpOnly e SameSite=Strict, com Secure em produção. Consultas parametrizadas e filtros por proprietário protegem os registros. A API limita corpo e tentativas de autenticação e valida entradas e origem.

## Integração demonstrada

Cadastro: formulário → validação e hash → INSERT usuarios. Login: verificação da senha → sessão/JWT → cookie. Meta: formulário autenticado → UPDATE metas e INSERT progresso_meta na mesma transação → consulta do histórico → gráfico/tabela. Comparação: cálculo dos cenários → INSERT juros e resultados associados → consulta no painel.

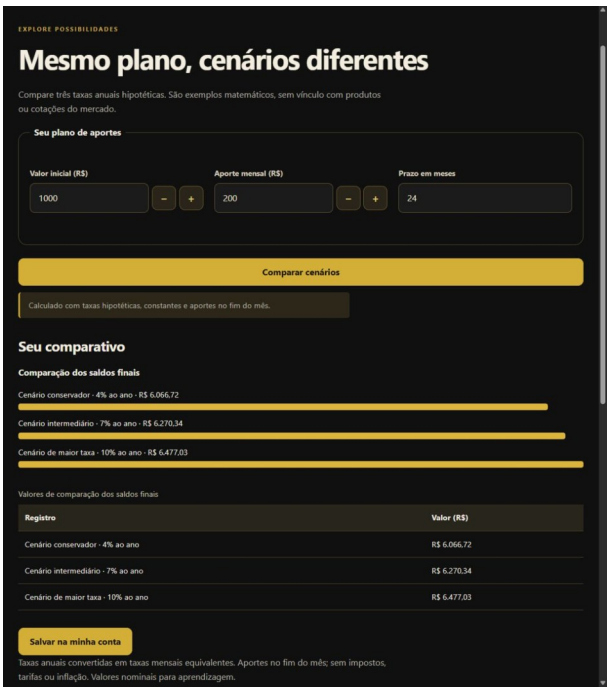



Figura 4 — Comparação salva em teste integrado com MySQL e dados fictícios. As taxas são hipóteses didáticas.

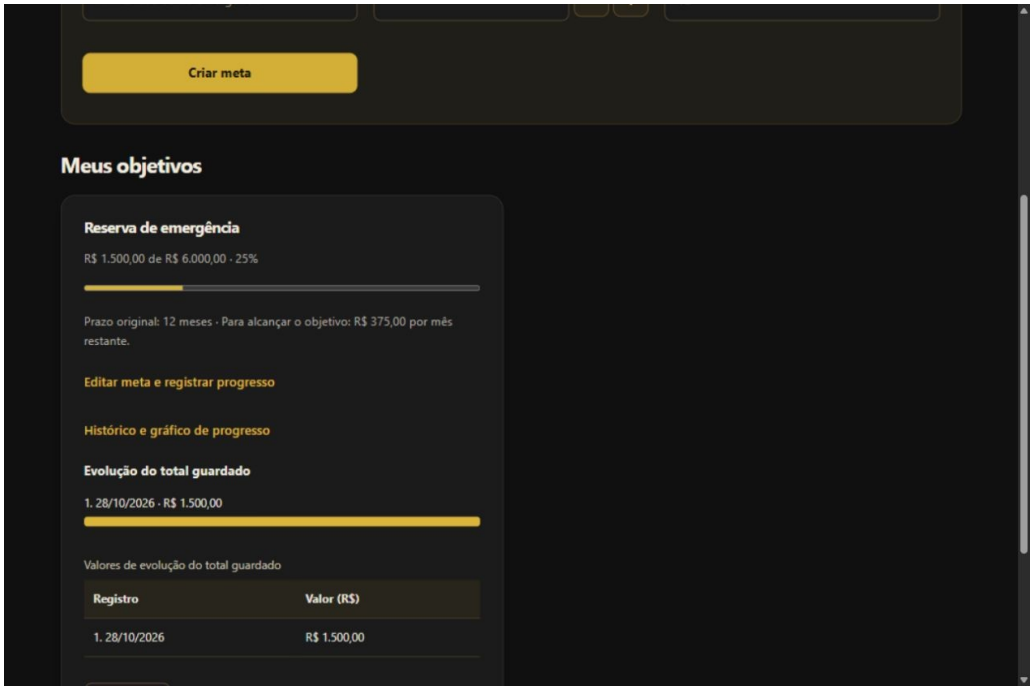



Figura 5 — Meta e progresso consultados pela API em teste integrado com MySQL. Dados fictícios de demonstração.

## Aplicação das boas práticas das aulas

Foram confrontados os 15 PDFs fornecidos, totalizando 312 páginas, com o código e os critérios da atividade. Os princípios aplicáveis foram incorporados sem tratar exemplos didáticos como requisitos obrigatórios de infraestrutura.

| Aulas e foco | Aplicação verificada / decisão |
|---|---|
| 01 — problema, público e ODS (22 p.) | Público de 16–30 anos, educação gratuita, ODS 4 principal e contribuição indireta à ODS 8; sem promessa de renda. |
| 02 — ciclo, Scrum e Kanban (32 p.) | Backlog por prioridade, definição de concluído, quadro e registro datado da revisão. Reuniões anteriores não comprovadas não são declaradas. |
| 03 — UI/UX (17 p.) | Jornada completa, identidade preto/dourado, estados vazios, feedback, prevenção de exclusão acidental e navegação consistente. |
| 04 — arquitetura (25 p.) | Monólito em camadas, Express/API/controllers/adaptadores SQL; decisões e diferenças documentadas. |
| 05 — Git e organização (18 p.) | Código formatado, README operacional, dependências fixadas, diretórios de servidor/SQL/testes/docs e .gitignore de dados privados. |
| 06 — HTML e SEO (28 p.) | HTML5, idioma, UTF-8, viewport, títulos e descrições; conta marcada noindex; landmarks e texto alternativo. |
| 07 — semântica textual (17 p.) | Hierarquia h1/h2/h3, parágrafos e conteúdos por nível com referências oficiais. |
| 08 — listas e tabelas (22 p.) | Menu em lista; tabela de dados junto dos gráficos, caption, thead/tbody e cabeçalhos scope. Não se usa tabela para layout do site. |
| 09 — formulários (15 p.) | Labels, required, tipos, limites, agrupamento em fieldset no comparativo; credenciais em POST; validação no servidor. |
| 10 — CSS (19 p.) | Estilo externo, variáveis, fonte de sistema, foco visível e contraste; remoção do CSS inline do orçamento. |
| 11 — layout e responsividade (23 p.) | Grid/Flex, controles de toque, menu mobile, tabelas com área de rolagem. Novos componentes com base mobile e expansão por min-width. |
| 12 — JavaScript (17 p.) | Módulos, const/let, funções reutilizáveis, conversão numérica explícita, regras de domínio e limites verificados. |
| 13 — DOM e eventos (25 p.) | addEventListener, preventDefault, input/change, DOM com textContent, aria-invalid e avisos de operação. |
| 14 — Storage (11 p.) | sessionStorage somente para preferências de categoria e nível; contas, metas e simulações no SQL. |
| 15 — JWT (21 p.) | JWT real assinado/expirado/verificado no servidor e sessão revogável. Cookie HttpOnly; o exemplo didático de token simulado em Storage não é usado como segurança. |

### Biblioteca e referências

A biblioteca reúne 15 resumos: cinco básicos, cinco intermediários e cinco avançados, distribuídos por 13 categorias. Inclui orçamento, reserva, inflação, crédito, diversificação, metas, segurança, renda fixa, custos e riscos. Cada artigo identifica as fontes oficiais e a data de consulta; busca textual e filtros podem ser combinados.

## 6. Testes

Na revisão de 28/10/2026, dez testes locais foram aprovados. A integração MySQL foi executada separadamente e aprovada; na execução comum ela é ignorada quando não configurada. Os testes usam dados fictícios e banco isolado, evitando alterar contas reais. Os resultados não constituem auditoria de segurança de produção.

| Verificação | Resultado e alcance |
|---|---|
| Autenticação e autorização | Cadastro/login/logout, JWT e isolamento entre contas aprovados. |
| CRUD e persistência | Orçamento, juros, metas e comparações; leitura após reabertura aprovada. |
| Transações MySQL | Rollback e relacionamentos/cascatas comprovados no teste isolado. |
| Cálculos e validação | Aportes ao fim do mês, taxa zero, limites e entradas inválidas verificados. |
| HTTP e formulários | JSON inválido, tamanho, origem e UTF-8; envio de credenciais protegido. |
| Migração local | Todos os campos originais comparados antes do commit; SQLite preservado. |
| Responsividade | Oito páginas verificadas em 320, 390, 768 e 1280 px na revisão; biblioteca ampliada reconferida nessas larguras. |
| Aprendizagem | 15 conteúdos, cinco por nível, fontes HTTPS, busca, filtros e estado vazio verificados. |

### Casos para reprodução

Juros: R$ 1.000 iniciais, R$ 200 ao fim de cada mês, taxa de 0,5% mensal e 24 meses resultam em R$ 6.213,55; o valor investido é R$ 5.800 e os juros, R$ 413,55. Meta: alvo de R$ 6.000 em 12 meses e saldo inicial de R$ 1.500 indicam 25% e aporte de R$ 375, enquanto restarem 12 meses.

Consultar docs/evidencias/testes.json, mysql.json, endpoints.json, banco.json e biblioteca-breakpoints.json. As evidências SQLite documentam os testes dessa alternativa; mysql.json documenta o esquema MySQL. As capturas com dados fictícios demonstram o sistema em execução. Não foram realizados testes em aparelho físico ou uma auditoria completa com leitores de tela.

## 7. Implantação

A implantação desta entrega é local. Na instalação demonstrada, INICIAR-FINSTART.cmd prepara e verifica o MySQL exclusivo e o servidor web, abrindo http://127.0.0.1:3000. Abrir index.html como arquivo não inicia a API; por isso os formulários são bloqueados nesse modo e orientam a abertura correta.

Em outro computador: instalar Node.js 24+ e MySQL Server compatível; instalar as dependências; executar a configuração local com o site parado; iniciar a aplicação. Para usar um servidor MySQL administrado pelo grupo, criar banco e usuário próprios e configurar o .env. O README descreve as duas opções. O pacote sem configuração privada inicia em SQLite; a opção MySQL requer preparação explícita.

O Workbench é um cliente de administração. Abrir schema.mysql.sql mostra o código, mas não cria conexão nem executa as tabelas automaticamente. A conexão da instalação local usa 127.0.0.1, porta 3308 e banco finstart. A senha privada fica somente na pasta data da máquina, fora do relatório, Git e ZIP.

### Backup e compartilhamento

O utilitário de backup exporta SQLite ou MySQL conforme o driver configurado. O backup MySQL usa mysqldump com transação consistente. Backups e credenciais devem ficar privados. O Git e o ZIP compartilham código, SQL, testes, documentos e evidências, não contas reais ou diretórios do banco.

Não há publicação Firebase, banco centralizado em nuvem ou HTTPS público nesta versão. A publicação do código depende de acesso de colaborador ao repositório do grupo. O envio no Canvas e a apresentação oral devem ser feitos pelo grupo após conferir os arquivos.

## 8. Manutenção / Evoluções Futuras

### Evolução em relação à segunda entrega

O protótipo anterior concentrava formulário inicial, página institucional e orçamento. A versão atual acrescenta fluxos completos de cadastro/login, biblioteca, juros,

comparação, metas, progresso e gráficos. A API, autenticação e persistência passaram de planejamento para implementação integrada, mantendo a arquitetura Node.js/Express/MySQL definida na segunda entrega.

| Problema observado | Correção aplicada |
|---|---|
| Telas do MVP incompletas | Oito páginas e fluxos de leitura, simulação e acompanhamento. |
| Banco/API/JWT apenas planejados | Controllers, consultas reais, JWT verificado e MySQL operacional. |
| HTML local sem executar módulos/API | Inicializador HTTP e bloqueio orientado de file://. |
| Senha aparecendo na URL | Formulários POST explícitos e proteção antes dos módulos. |
| Saldo sem histórico e prazo | Registro transacional, prazo e cálculo mensal necessário. |
| Conteúdo apenas introdutório | Biblioteca com três níveis e fontes oficiais. |
| Kanban sem trabalho em andamento | Quadro atual datado, com critérios de conclusão e pendências reais. |

Próximos passos: recuperação de senha e verificação de e-mail; testes com usuários e acessibilidade assistiva; hospedagem com HTTPS e backup externo; acompanhamento de erros; revisão periódica das fontes; eventual integração com dados reais. Produtos financeiros e rentabilidades atuais não devem ser confundidos com os cenários hipotéticos existentes.

# 9. Considerações Finais

A FinStart mantém a proposta educativa original e apresenta integração entre interface, API e banco relacional. A consolidação está demonstrada pelos fluxos implementados, scripts SQL, modelo atualizado e verificações registradas. O grupo deve revisar o pacote e conseguir explicar os cálculos, as relações entre tabelas e os limites da versão durante a apresentação.

| Tópico exigido na 3ª entrega | Localização neste documento |
|---|---|
| 1 Front-end implementado | Seções 2, 4 e 5; capturas das telas. |
| 2 Back-end funcionando | Seções 5 e 6; rotas, controllers e testes. |
| 3 Banco consolidado | Seção 5; DER, tabelas, FKs, SQL e conexão. |

| Tópico exigido na 3ª entrega | Localização neste documento |
|---|---|
| 4 Integração real | Seções 5 e 6; fluxos completos e persistência. |
| 5 Evolução desde a 2ª entrega | Seção 8; alterações, desafios e próximos passos. |
| 6 Clareza e organização | Modelo oficial, estrutura cumulativa e evidências identificadas. |

## Referências documentais e técnicas

Documentos do grupo: primeira entrega original e corrigida; 2a-Entrega-FinStart; Relatorio_Parcial_1_FinStart. Materiais da disciplina: aulas 01 a 15; comentários de Fernando Luiz de Almeida Silveira; enunciado da terceira entrega; documento padrão oficial fornecido pelo professor.

Documentação técnica: Node.js (https://nodejs.org/docs/latest-v24.x/api/), Express (https://expressjs.com/), MySQL (https://dev.mysql.com/doc/refman/8.0/en/) e SQLite (https://www.sqlite.org/).

Fontes educativas: Banco Central, CVM/Portal do Investidor, IBGE e Tesouro Direto. Os endereços específicos, títulos e data de consulta estão em cada conteúdo da biblioteca e no catálogo versionado server/conteudos.js. Consulta dos conteúdos: 28/10/2026.

Pendências de entrega: revisão final pelo grupo, publicação com permissão no GitHub, upload do PDF no Canvas e apresentação oral. A verificação técnica não garante nota nem substitui a avaliação do professor.